

FRC PROGRAMMING CURRICULUM — MODULE 1, LESSON 03

What Is a Subsystem?

Subsystem anatomy, the CommandScheduler, the IO pattern preview, building Scoop, tiered challenge, Broken Robot Lab.

Instructor Edition

Not for student distribution.

At a Glance

Field	Detail
Target audience	Students who completed Lesson 02
Hardware	None — VS Code and simulator only
Session length	3 hours
Key tools	WPILib: Create a new class, Simulator, AdvantageScope
Reference	BearBots_Program_Flow.pdf — 15-page flowchart + file-card reference. Keep it out during Part 2 and Part 4.

This guide follows the six parts of lesson-03-subsystems.html in order. Don't invent activities outside this structure — the digital handout is the source of truth for what students see.

Learning Objectives

- Students can explain why `periodic()` cannot contain motor control, using the `CommandScheduler`'s loop sequence as the reason
- Students can name the four parts of every subsystem (fields, constructor, `periodic()`, methods/commands)
- Students can read a real subsystem (`Arm.java`) and explain what each part does
- Students can explain, at a preview level, why a subsystem doesn't import hardware directly (the IO pattern, fully built in Lesson 04)
- Students can build a subsystem using the goal pattern: a command sets a goal, `periodic()` reads the goal and applies it
- Students can register a subsystem in `RobotContainer` and verify it loads in `AdvantageScope`
- Students can identify all four Broken Robot Lab bug types in unfamiliar code

Before You Start

Setup

- Lesson 02 project open in VS Code on projector
- Simulator confirmed working from Lesson 02
- AdvantageScope confirmed connecting from Lesson 02
- Digital handout open: [lesson-03-subsystems.html](#)
- BearBots_Program_Flow.pdf printed or open on a second screen for reference during Part 2 and Part 4

Constants pattern note

The curriculum teaches constants as separate files per subsystem (DriveConstants.java, ArmConstants.java, etc.), established in Lesson 02. If the working XRP codebase still uses the inner-class pattern in places, don't surface that mismatch to students — teach the separate-file pattern as correct and resolve the codebase divergence outside of class.

Session Timing

Time	Phase	You do	Students do
0–10 min	Hook	Side-by-side: monolithic Robot.java vs. structured subsystems. Ask which they'd rather debug at 11pm.	Look at both. Form opinions. Argue.
10–35 min	Part 1 — The Concept	Cafeteria analogy. Three reasons the structure earns its complexity. Four parts of a subsystem. Why periodic() cannot control motors — run the CommandScheduler simulator live (Good / Bad / Conflict).	Follow along. Run the three scenarios themselves. Answer the Part 1 quick check (Q1–Q3).
35–65 min	Part 2 — Reading a Real Subsystem	Walk Arm.java top to bottom using the annotated view. Point out the missing XRPServo import as the IO pattern teaser.	Read along. Answer “explain this out loud” before moving on.
65–85 min	Part 3 — The IO Pattern (preview)	Show ArmIO.java, ArmIOXRP.java, ArmIOSim.java side by side. Show how RobotContainer picks the implementation by mode. Frame this explicitly as a Lesson 04 preview, not a build-it-now skill.	Follow along. No coding yet — this part is conceptual.
85–145 min	Part 4 — Build Scoop	Live demo the 7-step build (file → field → enum → @AutoLogOutput → setGoal()/setGoalCommand() → wire periodic() → register + verify).	Replicate each step on their own laptop, in sync with the demo. Build, simulate, verify in AdvantageScope after every step.
145–165 min	Part 5 — Tiered Challenge	Circulate. Ask “what do you expect?” before every run.	Bronze: verify Scoop. Silver: build Elevator with the snippet + goal-pattern diff challenge. Gold: design the

Time	Phase	You do	Students do
			full robot architecture.
165–175 min	Part 6 — Broken Robot Lab	Circulate. Don't give answers — give direction. Make sure all 4 bugs are correctly explained, not just located.	Click through all 4 bugs. Read the reveal and fix for each.
175–180 min	Connect + wrap	Tie Scoop back to Competition Connection (Drive, Elevator, Scoop, Arm). Tease Lesson 04.	Recognize the pattern in the real robot's four subsystems.

If running short

Part 3 (IO Pattern preview) can compress to 10 minutes — it's intentionally conceptual, full build comes in Lesson 04. Never cut Part 6; the four bugs are the most diagnostic moment in the lesson for whether the `periodic()`-vs-commands distinction actually landed.

Phase 0 — Hook (0–10 min)

Side-by-side comparison

Prepare two projects before class:

- **Project 1 — monolithic:** A Robot.java with 600+ lines containing drivetrain, shooter, intake, and autonomous all in one file.
- **Project 2 — structured:** The same robot behavior split into proper subsystems. Each file is under 100 lines.

Open both on the projector. Scroll slowly through the monolithic version. Count the lines out loud. Then ask:

Script

"It's 11pm. Your autonomous isn't working. Which of these would you rather debug?"

Don't explain subsystems yet. Let the comparison create the need.

Part 1 — The Concept (10–35 min)

The cafeteria analogy

Script

"Imagine your school cafeteria. Pizza station, salad bar, dessert section. Each one does its job. If the pizza oven breaks, the ice cream is fine. Your robot works the same way — each subsystem owns one mechanism. If the scoop breaks, the drivetrain keeps driving."

The HTML pairs this with four cards: Drive, Elevator, Scoop, Arm — the BearBots robot's actual four subsystems. Point at each card as you say its name; this is the first time students see all four named together.

Three reasons the structure earns its complexity

Walk all three — don't skip to the third one early, even though it's the "WPILib enforces it" reason that feels most concrete:

1. One job, one place — debugging at 2am, you check one file
2. Teammates work in parallel — different files, no merge conflicts
3. WPILib enforces it — the CommandScheduler won't let two commands fight over the same subsystem

The four parts of a subsystem

Every subsystem has the same anatomy. Teach this as a checklist students can apply to any subsystem they read for the rest of the program:

4. Fields — hardware objects, goal state, current state. Private.
5. Constructor — runs once at startup. Configure hardware, set initial state.
6. periodic() — runs every 20ms, forever. Sensors and logging only.
7. Methods & Commands — the public API. Often return Command objects.

Why `periodic()` cannot control motors — run the simulator live

This is the conceptual core of the lesson. Use the CommandScheduler terminal simulator widget on the projector.

The mechanism, precisely

Each 20ms loop, the scheduler runs: `periodic()` for every subsystem first → poll triggers/buttons → run active commands. `periodic()` runs unconditionally — the scheduler cannot skip, delay, or interrupt it. It has no requirements, so the scheduler's conflict resolution can't touch it. Commands are the opposite: they can be started, stopped, interrupted, coordinated.

Run all three scenarios on the projector, in order:

- **Good** — `periodic()` reads sensors only; a drive command runs for several cycles and ends cleanly. Scheduler controls the full lifecycle.
- **Bad** — motor output lives in `periodic()`. The scheduler calls it at step 1, unconditionally, every loop, forever. A disable has no effect on it.
- **Conflict** — two commands both require the same subsystem. The scheduler resolves this cleanly; this is not the same failure mode as the Bad scenario, and students often conflate the two.

Common misconception to correct directly

Students may assume motor-in-`periodic()` causes “last one wins” behavior like a command conflict. It does not — it's undefined, hardware-dependent behavior because the scheduler has no visibility into `periodic()` at all. Don't let “last one wins” framing stand uncorrected.

Quick check — before Part 2

- **Q1** — Which of these belongs in `periodic()`? (Reading sensors / logging — correct answer. Motor control is the trap answer.)
- **Q2** — BearBots has an elevator that lifts a scoop. One subsystem or two? (Two — each owns one mechanism; coordination lives in commands, not by merging the subsystems.)
- **Q3** — What happens if you don't instantiate a subsystem in RobotContainer? (The class exists on disk but the object is never created — `periodic()` never runs, and the subsystem folder won't appear in AdvantageScope. This is a debugging dead end many students will hit later; flag it now.)

Part 2 — Reading a Real Subsystem (35–65 min)

Walk Arm.java using the annotated view, group by group, in this order:

8. Imports — point out Pose3d/Rotation3d (WPIlib geometry) and LoggedMechanism2d (AdvantageKit visualization). Then stop and ask: “What import is missing here?” There is no XRPServo import. This is the IO pattern teaser — the subsystem doesn’t know what hardware it’s running on.
9. Class declaration + fields — extends SubsystemBase is the one line that registers with the scheduler. ArmIO io is the IO pattern in miniature: an interface, not a hardware object. ArmIOInputsAutoLogged is compiler-generated from @AutoLog.
10. Constructor — three lines, dependency injection. RobotContainer decides which ArmIO implementation gets passed in; Arm itself never changes.
11. periodic() — io.updateInputs(inputs) + Logger.processInputs(“Arm”, inputs) is the heartbeat every AdvantageKit subsystem shares. Everything else in that loop cycle reads from the same frozen inputs snapshot.
12. Methods — setAngle() is the gatekeeper for hardware access; commands call it, they never touch io directly. stop() moves to a safe stowed position — for a servo, “stopped” means holding a safe angle, not going limp.

Before moving on — explain this out loud

Without looking: what are the four sections of Arm.java? Why does periodic() call io.updateInputs() before anything else? What would break if it didn’t? Cold-call a student rather than asking for volunteers — this is the moment to confirm the mental model actually landed before Part 3 builds on it.

Part 3 — The IO Pattern, preview only (65–85 min)

Framing — say this explicitly before starting

“This part is a preview. You will build the full IO pattern in Lesson 04. Today, just understand why it exists — you’ll build Scoop the simple way on purpose.”

Show three files side by side on the projector:

- **ArmIO.java** — the interface. Defines @AutoLog class ArmIOInputs and the operations (updateInputs, setAngle) any arm hardware must support, with no hardware specifics.
- **ArmIOXRP.java** — the real implementation. Imports XRPServo, knows about XRP hardware specifically.
- **ArmIOSim.java (optional)** — the simulation stub. No hardware imports at all.

Then show how RobotContainer picks the implementation based on Constants.currentMode (REAL vs SIM). The subsystem itself never decides — that decision lives entirely outside Arm.java.

Don’t over-teach this part

Students are not implementing IO classes today. The goal is recognition: “a subsystem talks to an interface, not hardware directly” — full implementation skill comes in Lesson 04.

Part 4 — Build Scoop (85–145 min)

Live demo, in sync, step by step. This is the longest block of the session — protect the time.

The goal pattern, taught explicitly before starting:

Script

“Commands set the goal. `periodic()` reads the goal and applies it to hardware. The servo always tracks the current goal — if the goal doesn’t change, neither does the servo. This is different from calling `motor.set(0.5)` directly inside `periodic()`, which pushes power regardless of any command state.”

The 7 steps

13. Create the file with the WPILib tool (SubsystemBase) — never by hand. Right-click subsystems/ → Create a new class, or Ctrl+Shift+P → WPILib: Create a new class.
14. Add the hardware field — `XRPServo scoopServo = new XRPServo(4)`.
15. Add a Goal enum — FLAT, CARRY, DUMP.
16. Add `@AutoLogOutput` on the goal field so it shows in AdvantageScope.
17. Add `setGoal()` and a `setGoalCommand()`.
18. Wire `periodic()` to read the current goal and apply it to the servo — this is the step students most often get backwards. Watch for students putting the servo call inside the command instead of `periodic()`.
19. Register in RobotContainer, bind a button, build → simulate → verify in AdvantageScope.

Verification habit — teach this as a standing rule, not just for today

After every structural change: (1) build succeeds, (2) sim launches with no console errors, (3) subsystem folder appears in AdvantageScope’s log tree, (4) goal changes when the button is pressed. If the folder doesn’t appear: check the subsystem is actually instantiated in RobotContainer, not just declared (echo of Q3 from Part 1).

Part 5 — Tiered Challenge (145–165 min)

Circulate. Ask “what do you expect to happen?” before every sim run — don’t let students run-and-check without predicting first.

Bronze — verify Scoop

Students confirm their Part 4 build completely: build succeeds, Scoop/ appears in AdvantageScope, D-pad up/down/right correctly change Goal between CARRY/DUMP/FLAT, and a print statement in periodic() confirms the loop is actually running.

If stuck

Check scoop is declared and assigned in RobotContainer (the object must be created, not just the class existing). Check the button binding in configureButtonBindings() and that the sim is enabled in Teleop.

Silver — add Elevator

Students use the aksubsystem snippet to scaffold Elevator (motor + encoder, not a servo). They add an ElevatorGoal enum (STOWED, LOW, HIGH), wire periodic() to apply a fixed speed per goal, add @AutoLogOutput, and register + verify.

The diff challenge (Option A vs Option B): Option A runs the motor forever at a fixed 40% regardless of goal — it never stops. Option B switches on the goal and applies the correct speed per state, including 0 for STOWED. Option B is correct — same goal pattern as the Scoop servo.

Port assignment gotcha, built into the activity on purpose

The prompt deliberately says “wait, is port 1 correct? Check the port assignments” — this is intentional friction, not a typo. Make students actually check rather than copy-paste blind.

Gold — design the full robot architecture

No code yet. Students list every mechanism on the BearBots robot, define hardware/goal-states/commands for each, and identify coordination scenarios (elevator rises → scoop must tilt up). Critical teaching point: coordination lives in commands that require multiple subsystems, not inside either subsystem’s own code. Don’t rush Gold students toward implementation — the design document is the deliverable.

Part 6 — Broken Robot Lab (165–175 min)

Four bugs, all in a Scoop class. Students click each highlighted line to reveal the bug.

Bug	What's wrong	Fix
1	Scoop servo declared inside Drive	Move scoopServo to Scoop.java
2	Motor control in periodic() — drives forward unconditionally	Move to a default command; periodic() reads sensors/logs only
3	tiltScoop() defined inside Drive, so it requires Drive instead of Scoop — locks out driving while tilting	Move tiltScoop() to Scoop, where it correctly requires only the scoop
4	emergencyStop() sets motors to 0 in a runOnce(), but periodic() overrides it back to 0.5 on the very next loop (~20ms later)	Fix Bug 2 first, then implement a STOPPED goal state that periodic() reads and applies

Bug 4 is the payoff bug

It's a second-order consequence of Bug 2 — make sure students articulate why fixing Bug 2 first is required before Bug 4 can actually be fixed. This is the moment the periodic() rule stops being abstract and becomes “this is why your emergency stop didn't work.”

Connect + Wrap (175–180 min)

Competition Connection

Script

“The BearBots robot has four subsystems: Drive, Elevator, Scoop, Arm. Each owns exactly one mechanism. If the scoop breaks, the drivetrain keeps driving. If the elevator breaks, the arm still works. That independence is why the robot can compete even when something goes wrong.”

Teaser for Lesson 04

Script

“Today you built Scoop the simple way — direct hardware field, no IO layer. Next lesson, you'll find out why two files exist for one motor, and build the full IO pattern from scratch: the interface, the real implementation, and the simulation stub. Same goal pattern you used today — just with one more layer underneath it.”